JAVA

Introduction to JAVA:

Java is a technology used to develop applications such as

- Standalone/Desktop Applications
- Distributed Applications
- Mobile Applications etc.,

Desktop Applications:

- These applications are standalone applications that run on the local machine.
- These applications won't run on Browser or Server.
- We can not share information through the Internet.
- Examples of Standalone Applications are Calculator, Notepad, Ms.Office, etc.,

Distributed Applications:

- These applications run on both Browser and Server
- We can share any type of information through the Internet.
- Examples of Distributed Applications are G-mail, Facebook, etc.,

Java was developed by Sun Microsystems in the year 1994 and it was released into the industry in the year 1995. The father of Java software is James Gosling. Nowadays Java is acquired by Oracle Corporation.

Java is mainly classified into 3 modules.

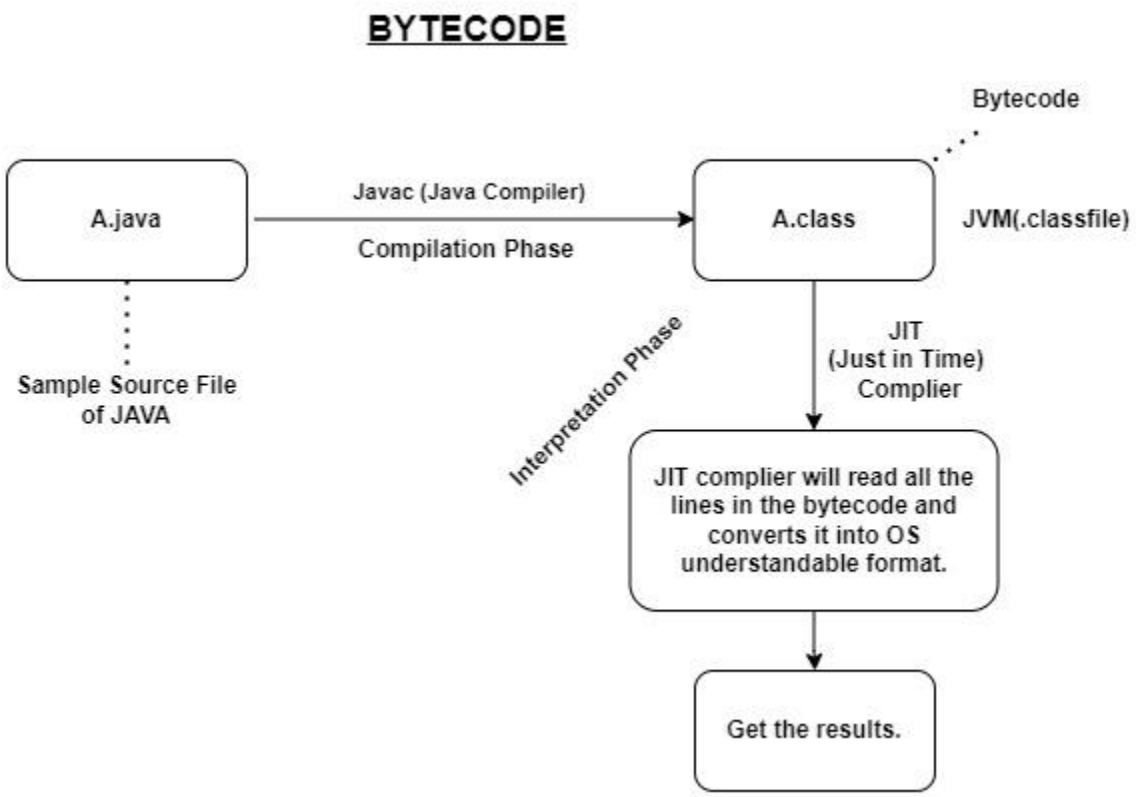
- **JSE [Java Standard Edition]:** It is used to develop standalone or desktop applications.
- **JEE [Java Enterprise Edition]:** it is used to develop distributed applications.
- **JME [Java Micro/Mobile Edition]:** it is used to develop mobile applications.

Features of Java:

1. **Simple:**

Java is a simple programming language because of 2 reasons.

- **Generic API (Application Programming Interface)**
  - Sun Microsystems has provided an inbuilt facility to the users, an API which is a collection of inbuilt packages, A package is a collection of classes, interfaces, and sub-packages. These multiple classes and interfaces internally contain predefined variables, Methods, and Constructors.
  - By using this API, we can develop different kinds of Applications.
- **Bytecode:**



Platform Independent:

Java is a platform-independent programming language because whenever we create a Java file and compile it on any OS, we get a .class file. If we copy this

.class into any other OS, we will still be able to run and execute the program without any interruptions. Java satisfies the slogan "Write once run anywhere".

- Byte code is platform-independent, but JVM is not.

Architectural Neutral:

Java software will run on every processor (intel, AMD, M2, etc.,)

Portable:

Java is a portable programming language because it satisfies both platform-independent and Architectural neutral features of Java.

Dynamic:

Java is a dynamic programming language because of the concept called collection framework. By using this we can store different types of values. If we keep adding elements, the size will automatically increase and vice-versa. We can perform any changes easily by using predefined methods.

Distributed:

Java is a distributed programming language because of the ability to develop applications, which can run on both Browsers and servers.

Network:

Java is a Network programming language because of the ability to develop applications that can connect systems like client-server, client-multiple servers, server-database, and multiple servers-databases by using a network (IP address, DNS, URL, Protocol, etc.,) as a communication layer.

Multi-Threading:

Java is a multi-threading programming language because in every program we have two threads in common.

- Foreground thread
- Background thread

The main purpose of the Foreground thread is to execute the logic of the program and the main purpose of the Background thread is to take care of the

execution status of the Foreground thread. Multi-threading features will provide a parallel or concurrent flow of execution.

High performance:

Java is a high-performance language because of two reasons.

- Byte code
- Garbage collector

If there is any leakage of memory space in our program, then JVM will call the garbage collector automatically to deallocate all the unused memory space. Java software will take less time to execute a program and less time for application development.

Highly Interpreted:

Java is a highly interpreted programming language because of both JVM (Java Virtual Machine) and JIT (Just In Time) Compiler.

Robust:

Java is a robust programming language because of the presence of Exception Handling techniques, we can handle exceptions by using keywords like try, catch, finally, throw, and throws. By using exception handling we can make the program flow of execution normally without any interruptions.

Secured:

Java is a secured programming language because whenever we transfer data from client to server through the network, the data will be transformed into an encrypted format.

Object-Oriented Programming:

- Java is an Object-Oriented programming language because it satisfies every concept of OOP like Inheritance, Encapsulation, Polymorphism and Abstraction.
- Even if we don't create any object for our class JVM will automatically create its object for the superclass of all our classes which is

"java.lang.Object". This object will act as memory space for all our properties.

- In our class, every property will be available in terms of objects.
- By using wrapper classes, we can convert the primitive type of data into object type and vice-versa.

Data types:

The main purpose of data types is to allocate a particular amount of memory space in the main memory for respective values and it will identify the type of values that are being stored.

Classification of data types:

- **Primitive/fundamental data types:**

  By using this we can store only single values in a single variable.

  Example:

```
int a = 10;  ✔

int b=10,20; ✖
```

Primitive data types are sub-classified into

- **Integer category datatypes:**

  By using this we can store only numerical values and we have data types like **byte**, **short**, **int**, and **long**.

- **Floating point category datatypes:**

  By using this we can able to store decimal type of values as well as numerals, we have the data types like **float** and **double**.

- **Character category data type:**

  By using this we can store a single character value and we need to specify the character within single quotations (' '). We have a datatype named **char** to store these values.

- **Boolean category datatypes:**

  By using this we can store either true or false kind of information and we have a datatype named **boolean**.

- **Nonprimitive/Derived data types:**

  By using this we can store multiple values of the same data types in a single variable, but we can't store multiple values of different data types in a single variable.

Example:

```
int a[] = new int[2];    // creating and declaring the size of an array
a[0]=1;    //inserting elements in first index value
a[1]=2;    // inserting elements in second index value
a[3]=3;    //cannot insert a value in the index which is not present.
int b[]={2,3};    // not a good practice to declare arrays statically
int c[]={"string", 5.6, 10, true};    //can not insert values of different datatypes
                                      //into an array that is declared integer.
```

- **User-defined/Custom-defined data types:**

  By using this we can store multiple values of different datatypes in a single variable to deal with custom-defined datatypes in C language we have a concept called "Data Structures".

Example:

In the case of **C language**

```
struct students{
    int rollNumber;
    float studentMarks;
    char[] studentName;
}
```

In the case of **Java**, we have both classes and interfaces

```
class Employee{
    int employeeId;
    float employeeSalary;
    String employeeName;
}
```

Variables:

These are also known as identifiers that are used to identify or recognize a value that will be stored in the main memory and whose values might vary from time to time. Hence the name Variable.

Declaration and Initialization:

```
int a;  // Declaration only

a=10; // initialization
```

```
int a = 10; // Declaration and Initialization
```

Types of variables:

- Instance/Global variable
- Local variable
- Static variable

Example:

```
class A // user-defined Datatype
{

    int x = 10; // Instance/Global Variable

    static String y = "James"; // Static Variable

    Run | Debug
    public static void main(String[] args) {

        boolean z = true; // Local Variable

    }
}
```

Methods:

Methods are block statements or code, which are used to implement a specific task or operation. The main purpose of a method is to implement the re-usability of logic or functionality.

Syntax:

```
<ReturnType/NonReturnType> <Method Name>(<Parameters>)
{
    // Body / Block of statements
}
```

Types of Methods:

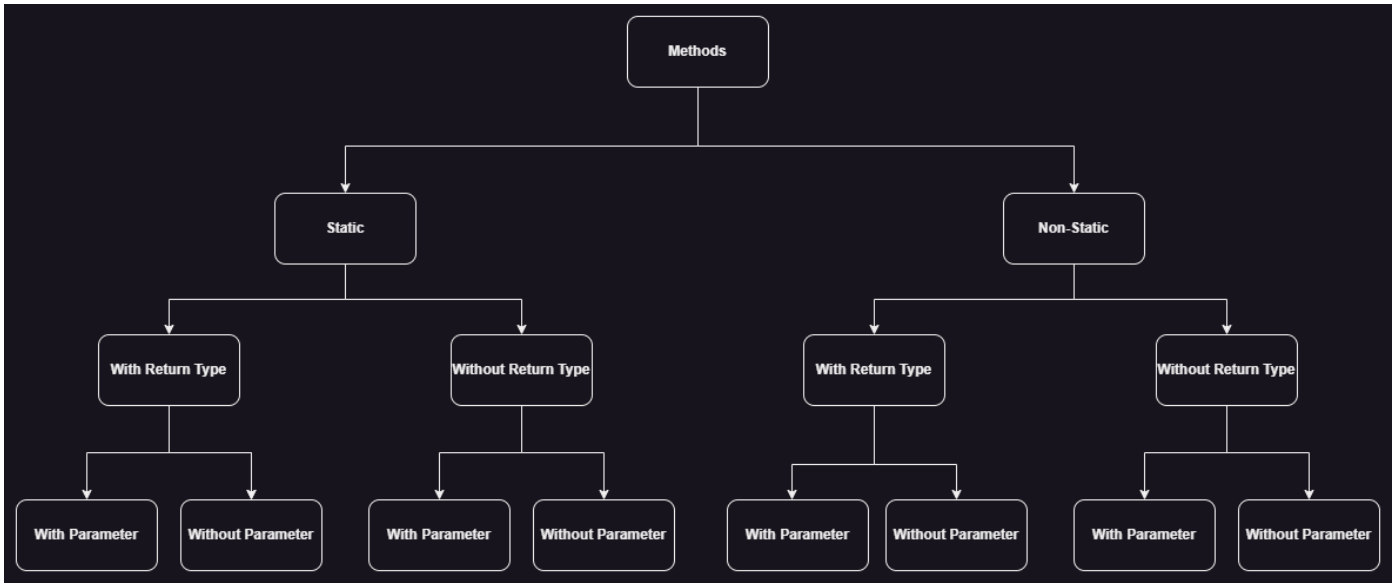
- **Predefined Methods:**

  These are inbuilt methods developed by Sun Microsystems and they deal with entire Java Software.

- **User-defined / Custom-defined Methods:**

  These methods are developed by programmers. They deal with specific projects or products. These user-defined methods are sub-classified into the following.

With Return Type	With Parameters	Static
Without Return Type	Without Parameters	Non-Static



Example of types of methods we can create:

```
static int m1(float a){
    return 10;
}
static int m2(){
    return 10;
}
static void m3(int b){

}
static void m4(){

}
```

```
int m5(float a){
    return 10;
}
int m6(){
    return 10;
}
void m7(int b){

}
void m8(){

}
```

Differences between Static and Nonstatic Properties:

Static Properties	Non-Static properties
The primary purpose of static properties is to perform common operations or one-time operations	The main purpose of non-static properties is to implement Unique (or)repeated operations.
We have to use a keyword called static for all static properties.	For all non-static properties, we shouldn't use static as a keyword.
Memory allocation will be done only once during compile time for all static properties	Memory allocation for all non-static properties will be done multiple times based on object creation.
It is not possible to create static variables under methods	It is possible to create non-static variables under methods.

We can invoke static properties under both static and non-static areas. We do not require an object to invoke them.	If we want to invoke non-static properties under a non-static area, we don't need an object but we need an object to invoke a non-static property under a static area.
Static properties can be accessed either through class name, object name, or directly.	Non-static properties can be accessed either through the object name or directly (in some cases only). But not through class name.
static methods can be overloaded but we cannot override a static method.	We can overload and override a non-static method.

Object Oriented Programming:

- Class
- Object
- Inheritance
- Data hiding
- Encapsulation
- Polymorphism
- Abstraction
- Dynamic binding
- Static binding
- Interfaces

Class:

It is a collection of variables, methods, and Constructors under a single unit or a single block. The primary purpose of a class is to declare user-defined datatypes.

- Class is a logical entity.
- The class contains state only (name, type, size, etc.,)
- Class does not have any behavior (actions or functionalities)
- We can create a user-defined class by using a predefined keyword called "class".

Syntax:

```
class <class name>
{
    // Variables declaration & initialization.

    // Methods declaration & initialization.

    // Constructors declaration & initialization.
}
```

Object:

It is an instance of a class, instance is nothing but the allocation of memory space for our class properties (methods, variables, constructors).

- Object is a real-time entity.
- Class is a blueprint from which individual objects are created.
- Class variable is called an object.

Syntax:

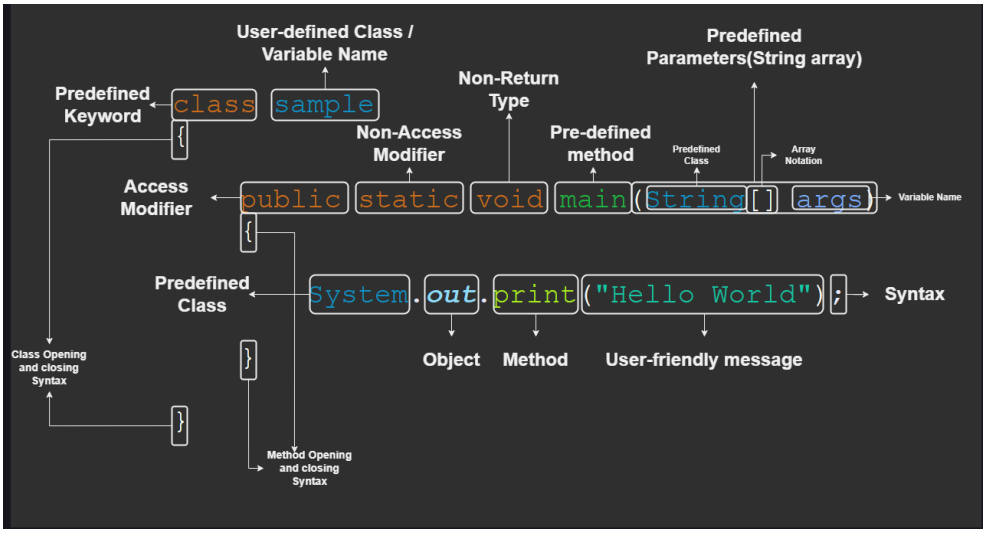
We can create an object for our user-defined class by using a predefined keyword called 'new'.

```
<class name> <object name> = new <class name>();

A obj = new A(); // Object creation by using a default constructor
```

We can create an object for a class in many ways, some of which are as follows.

- By using the 'new' keyword.
- By using the factory method.
- By using the clone method.
- By using deserialization.
- By using the new instance method.
- By using the get instance method.

Structure of Java program:

```
package details description;

import statement;

class <class name>
{
    // Variables declaration and initialization
    // Methods declaration and initialization
    // Constructors declaration and initialization

    public static void main(String[] args)
    {
        // Block of statements inside the main method
        --------
        -------- (Block of statements)
        --------
    }
}
```